

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

### CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

|                |                                                                                                                                                 |               |                                                    |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------|----------------------------------------------------|
| Course Code:   | CSA10 / CSA1063                                                                                                                                 | Course Title: | Software Engineering                               |
| CO Assessed:   | CO2                                                                                                                                             | Assessment:   | Assessment Tool 2 – Architecture Design Assignment |
| Weightage:     | 20%                                                                                                                                             | Total Marks:  | 25                                                 |
| CO2 Statement: | <i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i> |               |                                                    |
| Student Name:  | A. Govardhan Reddy                                                                                                                              | Reg. No.:     | 192421359                                          |
| Date:          | 07 – 08 - 2026                                                                                                                                  | Duration:     | 60 minutes                                         |

#### Code of Conduct

I A. Govardhan Reddy(192421359) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

### Annexure A – Architecture Design Assignment

#### Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

#### Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

#### 1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

#### 2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

#### 3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

#### 4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

#### 5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

#### 6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

#### Rubrics for Calculation

| Evaluation Criteria                                    | Excellent (4 Marks)                                                                                                                                                                | Good (3 Marks)                                                          | Satisfactory (2 Marks)                                         | Needs Improvement (1 Mark)                                        |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------|
| <b>System Requirement Analysis (30%)</b>               | Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture. | Analyzes most requirements with minor omissions or limited explanation. | Identifies only basic requirements with partial analysis.      | Requirement analysis is incomplete, inaccurate, or lacks clarity. |
| <b>Selection of Software Architecture (25%)</b>        | Selects the most appropriate architectural style with strong technical justification aligned to system requirements.                                                               | Selects a suitable architecture with reasonable justification.          | Architecture is partially suitable with limited justification. | Inappropriate architecture selected or justification is missing.  |
| <b>Architecture Diagram and Component Design (20%)</b> | Produces a clear, complete, and well-structured architecture diagram with all                                                                                                      | Diagram is mostly complete with minor omissions or notation issues.     | Diagram includes only essential components and                 | Diagram is incomplete, unclear, or                                |

|                                                                     |                                                                                                                                                                         |                                                                            |                                                                                                        |                                                                                   |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|                                                                     | major components, interfaces, and interactions accurately represented.                                                                                                  |                                                                            | lacks sufficient detail.                                                                               | technically incorrect.                                                            |
| <b>Component Interaction and Quality Attribute Analysis (15%)</b>   | Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability. | Explains most components and quality attributes with minor gaps.           | Provides limited explanation of interactions and addresses only some quality attributes.               | Component interactions and quality attributes are poorly explained or missing.    |
| <b>Architectural Justification and Technical Presentation (10%)</b> | Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.            | Provides adequate justification with minor gaps; report is well organized. | Limited justification with minimal discussion of trade-offs or future scope; report needs improvement. | Justification is weak or absent; report lacks organization and technical clarity. |

| Criteria                                                            | Mark |
|---------------------------------------------------------------------|------|
| System Requirement Analysis <b>(30%)</b>                            | /5   |
| Selection of Software Architecture <b>(25%)</b>                     | /5   |
| Architecture Diagram and Component Design <b>(20%)</b>              | /5   |
| Component Interaction and Quality Attribute Analysis <b>(15%)</b>   | /5   |
| Architectural Justification and Technical Presentation <b>(10%)</b> | /5   |
| <b>TOTAL</b>                                                        | /25  |

# 1. System Requirements Analysis

## 1.1 System Objectives

The platform aims to enhance safety and operational efficiency at airport surfaces by continuously monitoring runways, taxiways, aircraft, and ground vehicles. Its core objectives are to detect and prevent runway incursions in real time, provide situational awareness to Air Traffic Control (ATC) and ground crews, and support data-driven decision-making for surface movement management.

## 1.2 Functional Requirements

- Ingest real-time data from runway sensors, radar/lidar, ADS-B transponders, ground vehicle GPS trackers, and CCTV/surface movement radar.
- Detect potential runway incursions and aircraft/vehicle conflicts using rule-based and AI/ML models, and generate alerts within sub-second latency.
- Track and fuse the positions of aircraft, ground vehicles, and personnel on a unified digital map/twin of the airport surface.
- Notify ATC, pilots, and ground crew through role-specific interfaces (web dashboard, mobile app, cockpit alert interface).
- Incorporate weather and NOTAM (Notice to Airmen) data to adjust risk thresholds based on visibility and runway conditions.
- Maintain audit logs and generate compliance/incident reports for regulatory and safety review.
- Support role-based access control for controllers, ground staff, and airport operations managers.

## 1.3 Non-Functional Requirements

- Real-time performance: alert generation latency must be under one second for safety-critical events.
- High availability: the system must operate 24/7 with minimal downtime, as failures directly impact runway safety.
- Scalability: the system must handle traffic spikes at busy airports with many simultaneous aircraft/vehicle movements.
- Security: sensitive operational and flight data must be protected against unauthorized access and tampering.
- Reliability and fault tolerance: no single point of failure should disable incursion detection.
- Maintainability: individual modules must be upgradable (e.g., improving the ML risk model) without full-system downtime.

## 1.4 Key Quality Attributes

Given the safety-critical nature of the domain, the architecture must prioritize scalability (to handle variable air-traffic load), reliability and availability (continuous operation), performance (real-time alerting), security (protection of operational data), and maintainability (independent evolution of detection algorithms and integrations).

## **2. Selection of Suitable Software Architecture**

### **2.1 Selected Architecture: Hybrid — Event-Driven Microservices with Layered Organization**

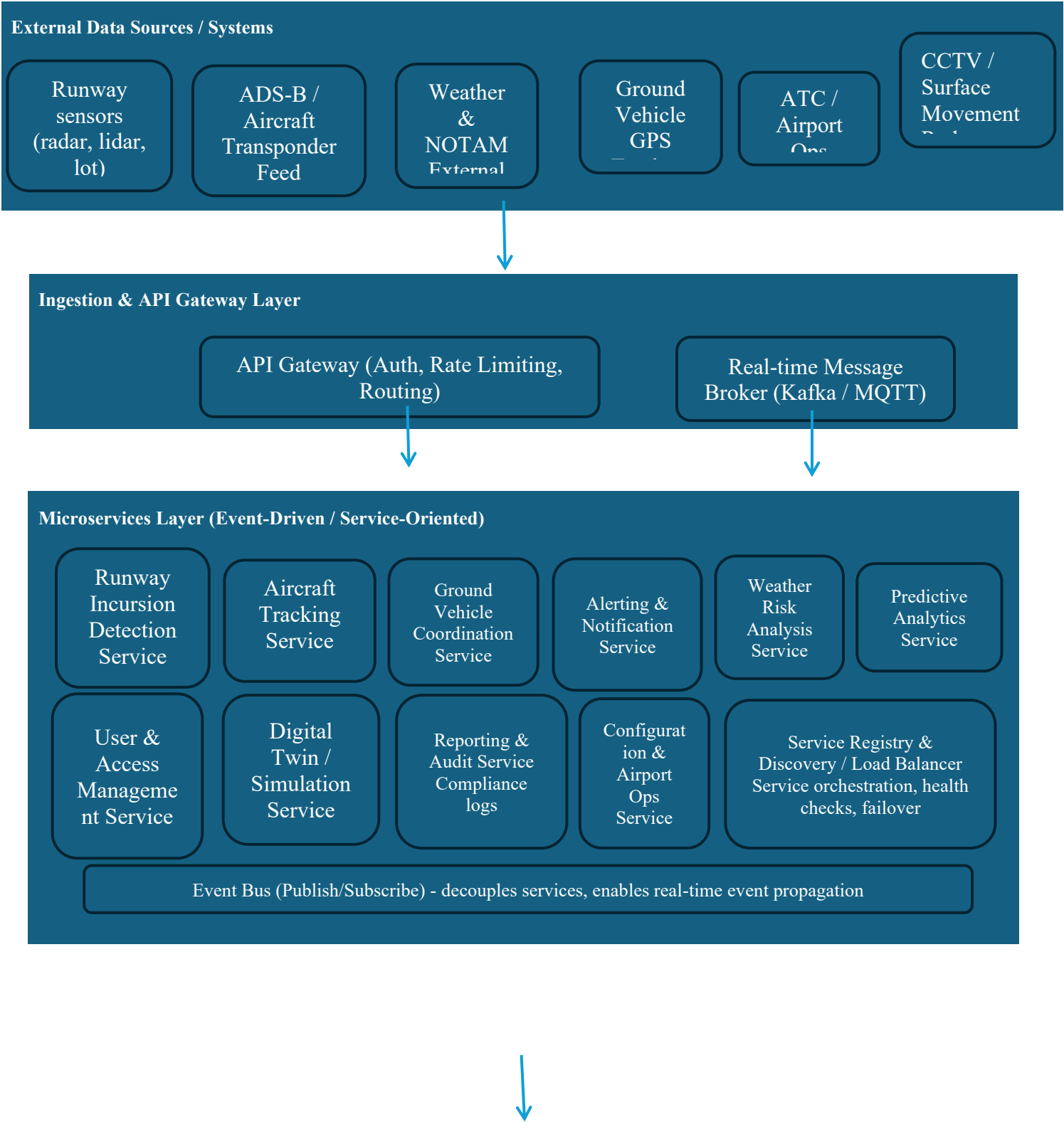
The proposed system adopts a hybrid architecture that combines a layered organization (ingestion, processing, data, presentation) with an event-driven microservices style at its core. Each functional capability — incursion detection, aircraft tracking, vehicle coordination, alerting, weather analysis, and predictive analytics — is implemented as an independent microservice communicating through an asynchronous event bus (e.g., Kafka/MQTT).

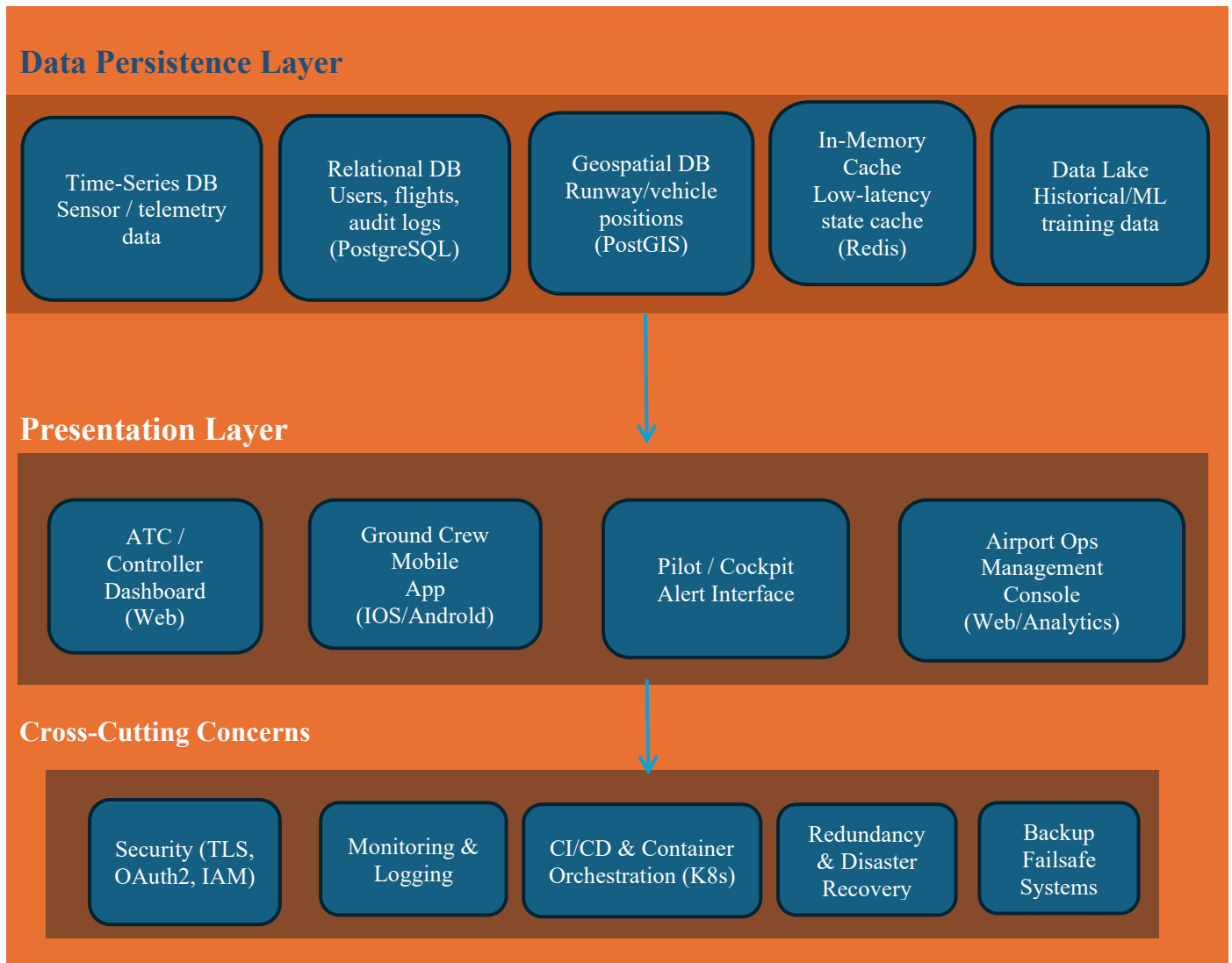
### **2.2 Justification**

- Event-driven communication is essential because runway safety data is inherently a continuous stream of real-time events (sensor readings, position updates, alerts) rather than discrete request/response transactions — a pure Client-Server or MVC style would not scale to this event volume with the required latency.
- A microservices/SOA approach isolates safety-critical logic (incursion detection) from less critical services (reporting, analytics), so a fault or update in a non-critical service cannot bring down the safety-alerting pipeline.
- The layered organization (external sources → ingestion → services → data → presentation) keeps the system understandable and enforces a clear separation of concerns, easing onboarding and maintenance.
- Compared to a monolithic or pure layered architecture, this hybrid approach allows independent scaling of high-load services (e.g., Aircraft Tracking during peak hours) without over-provisioning the entire system.
- Compared to a pure serverless approach, dedicated long-running microservices are more appropriate here because runway monitoring requires continuous, low-latency stream processing rather than short-lived, event-triggered functions.

### 3. Software Architecture Diagram

The diagram below illustrates the complete system architecture, organized into six layers: external data sources, ingestion/API gateway, the event-driven microservices layer (with an event bus), the data persistence layer, the presentation layer, and cross-cutting concerns spanning the whole system.





## 4. Components and Their Interactions

### 4.1 Component Responsibilities

- **External Data Sources:** Runway sensors, ADS-B feeds, weather/NOTAM APIs, ground vehicle GPS, ATC/SWIM systems, and CCTV/surface radar continuously generate raw operational data.
- **API Gateway:** Authenticates requests, applies rate limiting, and routes traffic to the appropriate backend services, acting as the single secure entry point.
- **Message Broker (Event Bus):** Decouples data producers from consumers; every sensor reading or status update is published as an event that interested services subscribe to.

- **Runway Incursion Detection Service:** Applies rule-based logic and AI models to identify potential conflicts between aircraft, vehicles, and runway status.
- **Aircraft Tracking / Ground Vehicle Coordination Services:** Fuse position data from multiple sources into a consistent, real-time surface-movement picture.
- **Alerting & Notification Service:** Consumes conflict/risk events and pushes prioritized alerts to ATC dashboards, pilot interfaces, and ground crew apps.
- **Weather Risk Analysis / Predictive Analytics Services:** Continuously assess environmental and historical risk factors to adjust incursion thresholds.
- **Data Persistence Layer:** Specialized stores — time-series DB for sensor streams, geospatial DB for positions, relational DB for users/audit data, Redis cache for low-latency lookups, and a data lake for ML training data.
- **Presentation Layer:** Role-specific interfaces — ATC dashboard, ground crew mobile app, pilot alert interface, and an operations management console.

## 4.2 Communication and Workflow

Sensor and tracking data flows from external sources through the API Gateway into the Kafka event bus.

Microservices subscribe to relevant event topics: the Aircraft Tracking and Ground Vehicle Coordination services continuously update position state, which the Incursion Detection Service consumes alongside weather-risk events to compute a real-time risk score. When a risk threshold is crossed, the service publishes an alert event that the Alerting & Notification Service delivers to the relevant dashboards and mobile/cockpit interfaces within sub-second latency. All events and decisions are simultaneously persisted to the appropriate databases for audit, reporting, and future ML model training via the data lake.

## 5. Discussion of Quality Attributes

| Quality Attribute | How the Architecture Addresses It                                                                                                                                                                                                                                                                                                                             |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scalability       | The microservices are independently deployable and horizontally scalable behind a load balancer and service registry. Kafka-based event streaming decouples producers (sensors, ADS-B feeds) from consumers, so ingestion throughput can scale independently of processing services during peak traffic (e.g., multiple simultaneous landings/take-offs).     |
| Security          | OAuth2/IAM-based authentication and role-based access control (RBAC) restrict dashboard and API access by role (ATC, ground crew, ops manager). TLS encrypts all data in transit; the API Gateway performs authentication, throttling, and input validation before requests reach internal services, reducing the attack surface on safety-critical services. |



|                        |                                                                                                                                                                                                                                                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Performance</b>     | A Redis in-memory cache serves low-latency lookups for aircraft/vehicle positions, and a dedicated time-series database (InfluxDB/TimescaleDB) is optimized for high-frequency sensor writes. The event-driven design ensures runway incursion alerts are generated and pushed within sub-second latency, critical for safety. |
| <b>Reliability</b>     | Redundant service instances behind the load balancer and Kafka's message durability guarantees prevent data loss if a service instance fails. Health checks and automatic failover in the service registry ensure the incursion detection pipeline stays operational even during partial outages.                              |
| <b>Maintainability</b> | Each microservice has a single, well-defined responsibility (e.g., Aircraft Tracking, Weather Risk Analysis), so teams can update, test, and redeploy services independently without affecting the rest of the system. Clear API contracts and a shared event schema keep integration overhead low.                            |
| <b>Availability</b>    | Multi-instance deployment across containers/Kubernetes with health monitoring (Prometheus/ELK) and automated restarts minimizes downtime. Disaster recovery and backup failsafe systems ensure continuity of the safety-alerting function, which cannot tolerate prolonged outages.                                            |

## 6. Justification of Architectural Decisions

### 6.1 Rationale

The hybrid event-driven microservices architecture was chosen because runway safety monitoring is fundamentally a real-time, high-throughput, safety-critical streaming problem. Decoupling services through an event bus ensures that a slowdown or failure in one non-critical service (e.g., analytics) does not compromise the safety-critical incursion detection and alerting pipeline, which is the system's primary responsibility.

### 6.2 Trade-offs Considered

- Complexity vs. resilience: microservices introduce operational complexity (service orchestration, distributed tracing) compared to a monolith, but this is justified by the fault isolation and independent scalability required for a safety-critical system.
- Eventual consistency vs. real-time responsiveness: the event-driven design favors low-latency propagation over strict transactional consistency; this is acceptable because stale data is refreshed within milliseconds and audit-critical writes still go through durable, ordered event logs.
- Operational cost vs. availability: running redundant service instances and multiple specialized databases increases infrastructure cost, but this is a necessary trade-off given that downtime in a runway safety system carries life-safety consequences.

### 6.3 Support for Future Enhancements and Long-Term Maintainability

- New capabilities (e.g., drone-detection or noise-monitoring services) can be added as new microservices that subscribe to the existing event bus without modifying core services.
- The predictive analytics and weather risk services can be independently retrained and redeployed as better ML models become available, without impacting the detection or alerting pipeline.
- Standardized APIs and event schemas ease integration with future airport systems (e.g., broader SWIM/ATC network upgrades) and support multi-airport deployment by replicating the service layer per site while sharing common infrastructure.
- The clear separation of layers and single-responsibility services keeps the codebase testable and maintainable as the system grows in scope and traffic volume.

# Rubric Self-Assessment Summary

| Criteria                                                     | Mark |
|--------------------------------------------------------------|------|
| System Requirement Analysis (30%)                            | /5   |
| Selection of Software Architecture (25%)                     | /5   |
| Architecture Diagram and Component Design (20%)              | /5   |
| Component Interaction and Quality Attribute Analysis (15%)   | /5   |
| Architectural Justification and Technical Presentation (10%) | /5   |
| TOTAL                                                        | /25  |